

Day 34–35: Generics

- Generic classes
- Generic methods

1. First: What problem do Generics solve?

Before understanding Generics, you need to understand a problem with normal Java collections.

Suppose we create an ArrayList:

```
ArrayList list = new ArrayList();
```

```
list.add(10);
```

```
list.add(20);
```

```
list.add("Hello");
```

```
list.add(30.5);
```

This is allowed.

The list now contains:

10

20

"Hello"

30.5

Problem

We wanted a list containing only integers, but Java allowed different types.

When we retrieve the values:

```
int x = (int) list.get(0);
```

We need **type casting**.

And if we accidentally do:

```
int x = (int) list.get(2);
```

we get:

ClassCastException

because "Hello" is a String, not an Integer.

So the question is:

Can we tell Java beforehand that this collection should contain only a particular type?

Yes.

That's where **Generics** come in.

2. What are Generics?

Simple definition

Generics allow us to specify the type of data a class, method, or collection can work with.

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

Here:

ArrayList

↓

<Integer>

↓

Only Integer values

So:

```
numbers.add(10);    // ✓
```

```
numbers.add(20);    // ✓
```

```
numbers.add(30);    // ✓
```

But:

```
numbers.add("Hello"); // ✗
```

```
numbers.add(10.5);   // ✗
```

Compiler itself tells us:

incompatible types

In simple words

Generics = **Type safety + reusable code**

3. Why is it called "Generic"?

"Generic" means **general / common / applicable to different types**.

Suppose we want a class that stores a value.

Without Generics:

```
class Box {  
    int value;  
}
```

This box can store only:

int

What if tomorrow we want a String box?

We might create:

```
class StringBox {  
    String value;  
}
```

What about Double?

```
class DoubleBox {  
    double value;  
}
```

This becomes repetitive.

Instead, we can create **one generic class**:

```
class Box<T> {  
    T value;  
}
```

Now the same class can work with:

Integer

String

Double

Character

Student

Employee

etc.

That's why it's called **Generic**.

4. What is T?

This is very important.

Look at:

```
class Box<T> {  
    T value;  
}
```

What is T?

T is a **type parameter**.

It is basically a placeholder for a data type.

Think:

T = "I don't know the type yet.

Whoever uses this class will tell me."

For example:

Box<Integer>

means:

T = Integer

So:

T value;

becomes conceptually:

Integer value;

Similarly:

Box<String>

means:

T = String

So:

T value;

becomes:

String value;

5. Generic class

A **generic class** is a class that uses a type parameter.

Basic syntax:

```
class ClassName<T> {
```

```
    T variable;
```

```
}
```

Example:

```
class Box<T> {
```

```
    T value;
```

```
    void setValue(T value) {
```

```
        this.value = value;
```

```
    }
```

```
    T getValue() {
```

```
        return value;
```

```
    }
```

```
}
```

Let's understand every part.

6. Understanding class Box<T>

class Box<T>

Break it:

class

↓

we are creating a class

Box

↓

class name

<T>

↓

type parameter

So:

class Box<T>

means:

"Create a class called Box that can work with some type T."

7. Generic variable

Inside:

T value;

Normally we might write:

int value;

or:

String value;

But we don't know the type yet.

So we write:

T value;

The actual type will be decided when the object is created.

8. Creating a Generic object

Suppose:

```
Box<Integer> box = new Box<>();
```

Here:

```
Box<Integer>
```

↑

T = Integer

Therefore:

T value;

effectively represents:

Integer value;

Now:

```
box.setValue(100);
```

works.

And:

```
box.setValue("Hello");
```

doesn't work.

9. Complete Generic Class Example

```
class Box<T> {
```

```
    private T value;
```

```
    void setValue(T value) {
```

```
        this.value = value;
```

```
    }
```

```
    T getValue() {  
        return value;  
    }  
}
```

Main:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Box<Integer> intBox = new Box<>();  
  
        intBox.setValue(100);  
  
        System.out.println(intBox.getValue());  
    }  
}
```

Output:

100

10. Step-by-step execution

Look at:

```
Box<Integer> intBox = new Box<>();
```

Java understands:

Box

↓

T is Integer

Then:

```
intBox.setValue(100);
```

Method:

```
void setValue(T value)
```

becomes conceptually:

```
void setValue(Integer value)
```

So:

```
100
```

is accepted.

Then:

```
intBox.getValue()
```

returns:

```
Integer
```

11. Generic class with String

Same class:

```
Box<String> stringBox = new Box<>();
```

```
stringBox.setValue("Hello");
```

```
System.out.println(stringBox.getValue());
```

Now:

```
T = String
```

So conceptually:

```
T value;
```

becomes:

```
String value;
```

Output:

```
Hello
```

12. Same class, different types

This is the main power of Generics.

```
Box<Integer> box1 = new Box<>();
```

```
Box<String> box2 = new Box<>();
```

```
Box<Double> box3 = new Box<>();
```

Think:

```
Box<Integer>
```

↓

```
T = Integer
```

```
Box<String>
```

↓

```
T = String
```

```
Box<Double>
```

↓

```
T = Double
```

One class:

```
Box<T>
```

can handle multiple types.

13. Real-world example of Generic Class

Imagine a storage box.

You have:

Box

One box stores:

Book

Another box stores:

Laptop

Another box stores:

Mobile

Instead of designing separate classes:

BookBox

LaptopBox

MobileBox

we can design:

Box<T>

Then:

Box<Book>

Box<Laptop>

Box<Mobile>

Same structure, different data type.

14. Generic class with two type parameters

Generics are not limited to one type.

We can write:

```
class Pair<K, V> {
```

```
    K key;
```

```
    V value;
```

```
}
```

Here we have:

$K \rightarrow$ first type

$V \rightarrow$ second type

Example:

Pair<Integer, String>

means:

K = Integer

V = String

So:

K key;

V value;

becomes:

Integer key;

String value;

15. Complete Pair Example

```
class Pair<K, V> {
```

```
    K key;
```

```
    V value;
```

```
    Pair(K key, V value) {
```

```
        this.key = key;
```

```
        this.value = value;
```

```
    }
```

```
    void display() {
```

```
        System.out.println("Key: " + key);
```

```
        System.out.println("Value: " + value);
```

```
    }
```

```
}
```

Main:

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
Pair<Integer, String> student =  
    new Pair<>(101, "Rahul");  
  
    student.display();  
}
```

Output:

Key: 101

Value: Rahul

Here:

K = Integer

V = String

16. Common Generic Type Names

You will frequently see:

Letter Common meaning

T Type

E Element

K Key

V Value

N Number

S Sometimes String/second type

Important:

These letters are **conventions**, not strict rules.

You could write:

```
class Box<ABC>
```

but normally we use:

```
class Box<T>
```

because it's easier for other programmers to understand.

17. Generic Method

Now let's move to the second major topic:

Generic Methods

A **generic method** is a method that has its **own type parameter**.

Example:

```
public static <T> void display(T value) {
```

```
    System.out.println(value);
```

```
}
```

Look carefully:

<T>

comes **before** the return type:

```
public static <T> void
```

This tells Java:

"This method has its own type parameter T."

18. Why do we need Generic Methods?

Suppose we want to print different types.

Without a generic method, we might write:

```
void displayInt(int value) {
```

```
    System.out.println(value);
```

```
}
```

```
void displayString(String value) {
```

```
    System.out.println(value);
```

```
}
```

```
void displayDouble(double value) {  
    System.out.println(value);  
}
```

Three methods doing almost the same thing.

Generics allow us to write one method:

```
<T> void display(T value) {  
    System.out.println(value);  
}
```

Now:

```
display(10);  
display("Hello");  
display(10.5);  
display('A');
```

One method handles multiple types.

19. Generic Method Syntax

General syntax:

```
<type_parameter> returnType methodName(parameters) {  
  
}
```

Example:

```
<T> void display(T value) {  
  
}
```

Another:

```
<T> T getValue(T value) {  
  
    return value;  
}
```

```
}
```

20. Complete Generic Method Example

```
public class Main {  
  
    public static <T> void display(T value) {  
  
        System.out.println(value);  
  
    }  
  
    public static void main(String[] args) {  
  
        display(100);  
  
        display("Hello");  
  
        display(10.5);  
  
        display('A');  
    }  
}
```

Output:

100

Hello

10.5

A

21. How does Java understand T here?

When you call:

```
display(100);
```

Java determines:

T = Integer

When:

```
display("Hello");
```

Java determines:

T = String

When:

```
display(10.5);
```

Java determines:

T = Double

So the same method can work with different types.

22. Generic method returning a value

A generic method can also return T.

```
public static <T> T getValue(T value) {
```

```
    return value;
```

```
}
```

Use:

```
Integer x = getValue(100);
```

```
String y = getValue("Hello");
```

```
Double z = getValue(10.5);
```

Java determines the appropriate type.

23. Generic method with two types

We can have multiple type parameters:

```
public static <K, V> void display(K key, V value) {  
  
    System.out.println(key);  
  
    System.out.println(value);  
  
}
```

Call:

```
display(101, "Java");
```

Java understands:

K = Integer

V = String

Another call:

```
display("Name", "Rahul");
```

Now:

K = String

V = String

24. Generic Class vs Generic Method

This is extremely important for interviews.

Generic class

```
class Box<T> {
```

```
    T value;
```

```
}
```

Here T belongs to the **class**.

Generic method

```
public static <T> void display(T value) {  
  
}
```

Here T belongs to the **method**.

Think:

Generic Class

↓

T belongs to class

Generic Method

↓

T belongs to method

25. Very important difference

Suppose:

```
class Box<T> {
```

```
    T value;
```

```
    <U> void display(U data) {
```

```
        System.out.println(data);
```

```
    }
```

```
}
```

Here we have **two different type parameters**:

T → belongs to class

U → belongs to method

They are independent.

Example:

```
Box<Integer> box = new Box<>();
```

```
box.display("Hello");
```

Here:

T = Integer

U = String

So a class can have one type while its method can have another type.

26. Generic Class + Generic Method Together

Example:

```
class Box<T> {
```

```
    private T value;
```

```
    void setValue(T value) {  
        this.value = value;  
    }
```

```
    T getValue() {  
        return value;  
    }
```

```
    <U> void display(U data) {  
        System.out.println(data);  
    }  
}
```

Main:

```
public class Main {
```

```
public static void main(String[] args) {  
  
    Box<Integer> box = new Box<>();  
  
    box.setValue(100);  
  
    System.out.println(box.getValue());  
  
    box.display("Hello");  
  
}  
}
```

Here:

T = Integer

U = String

Output:

100

Hello

27. Why not simply use Object?

This is a very common interview question.

You may think:

```
class Box {
```

```
    Object value;
```

```
}
```

Because Object is the parent class of almost all Java classes.

Example:

Object value = 100;

and:

Object value = "Hello";

Both work.

But there is a problem.

When retrieving:

Object value = 100;

you need casting:

int x = (Integer) value;

With Generics:

Box<Integer> box = new Box<>();

Java already knows:

value = Integer

So type safety is provided at compile time.

28. Generics provide Compile-Time Type Safety

This phrase is extremely important.

Suppose:

ArrayList<Integer> numbers = new ArrayList<>();

Then:

numbers.add(10); // ✓

numbers.add(20); // ✓

numbers.add("Hello"); // ✗

The compiler catches the problem **before the program runs**.

That's called:

Compile-time type safety.

Without Generics, some type errors may only appear when the program executes.

29. Generics and Collections

This is where you'll use Generics most frequently.

You've already learned:

List

ArrayList

LinkedList

Vector

Set

HashSet

Map

HashMap

Generics are heavily used with them.

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

```
ArrayList<String> names = new ArrayList<>();
```

```
HashSet<Integer> numbers = new HashSet<>();
```

```
HashMap<Integer, String> students = new HashMap<>();
```

30. Understanding HashMap<Integer, String>

This is a beautiful example of multiple Generics.

```
HashMap<Integer, String> students;
```

Here:

Integer → Key type

String → Value type

So:

```
students.put(101, "Ravi");
```

```
students.put(102, "Anu");
```

works.

But:

```
students.put("101", "Ravi");
```

doesn't work because key must be:

Integer

not:

String

31. Generic syntax with collections

You can think of:

```
ArrayList<Integer>
```

as:

ArrayList

+

Integer type information

And:

```
HashMap<Integer, String>
```

as:

HashMap

+

Integer key

+

String value

32. Why do we use wrapper classes?

You may remember Java primitive types:

int

double

char

boolean

But with Generics, we normally use wrapper classes:

int → Integer

double → Double

char → Character

boolean → Boolean

So:

ArrayList<int>

is invalid.

Instead:

ArrayList<Integer>

is correct.

Why?

Because Java Generics work with **reference types**, not primitive types.

33. Autoboxing connection

Suppose:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

10 is an int.

But the list expects:

Integer

Java automatically converts:

int

↓

Integer

This is called:

Autoboxing

And when retrieving:

```
int x = numbers.get(0);
```

Java converts:

Integer

↓

int

This is called:

Unboxing

So Generics and wrapper classes are closely related to autoboxing/unboxing.

34. Can we use any class as a Generic type?

Yes.

For example:

```
class Student {
```

```
    String name;
```

```
    Student(String name) {
```

```
        this.name = name;
```

```
    }
```

```
}
```

Now:

```
ArrayList<Student> students = new ArrayList<>();
```

This means:

ArrayList

↓

only Student objects

Then:

```
students.add(new Student("A"));
```

```
students.add(new Student("B"));
```

This is completely valid.

35. Generic class with custom object

Example:

```
class Box<T> {  
  
    T value;  
  
    Box(T value) {  
        this.value = value;  
    }  
  
    void display() {  
        System.out.println(value);  
    }  
}
```

Then:

```
Student s = new Student("Ravi");
```

```
Box<Student> box = new Box<>(s);
```

```
box.display();
```

Here:

```
T = Student
```

So Generics aren't only for:

Integer

String

Double

They can work with your own classes too.

36. Important rule: Generic type must be a reference type

You cannot:

`Box<int>`

✗

You use:

`Box<Integer>`

Similarly:

`Box<double>`

✗

Use:

`Box<Double>`

37. Generic methods do not have to be static

You may see:

```
public <T> void display(T value) {  
  
}
```

This is a generic instance method.

Or:

```
public static <T> void display(T value) {  
  
}
```

This is a generic static method.

Both can be generic.

38. Why is <T> placed before void?

Look at:

```
public static <T> void display(T value)
```

Some beginners ask:

Why can't we write `void <T> display?`

Because Java's syntax specifies that the method's type parameters are declared **before the return type**.

Structure:

`public static`

↓

`<T>`

↓

return type

↓

method name

So:

`public static <T> void display(T value)`

is correct.

39. Generic method vs overloaded methods

Without Generics:

```
void print(int x) {  
    System.out.println(x);  
}
```

```
void print(String x) {  
    System.out.println(x);  
}
```

```
void print(double x) {  
    System.out.println(x);  
}
```

With Generic method:

```
<T> void print(T x) {  
    System.out.println(x);  
}
```

One method can handle multiple reference types.

40. But Generics don't mean "anything without rules"

This is an important point.

Consider:

```
<T> void display(T value)
```

It means:

T can represent different types.

But sometimes we want to restrict T.

For example:

"T must be a Number or a subclass of Number."

We can use a **bounded type parameter**.

```
<T extends Number>
```

Example:

```
public static <T extends Number> void display(T value) {
```

```
    System.out.println(value);
```

```
}
```

Now:

```
display(10);    // ✓
```

```
display(10.5);  // ✓
```

```
display(100L);  // ✓
```

But:

```
display("Hello"); // ✗
```

because String does not extend Number.

You'll encounter bounded generics later, but remember the basic idea.

41. Generics and inheritance

Suppose:

```
class Animal {  
}
```

and:

```
class Dog extends Animal {  
}
```

You might think:

Dog is an Animal

therefore:

List<Dog>

should be a:

List<Animal>

But Java does **not** treat it that way.

This is important:

List<Dog>

is not a subtype of:

List<Animal>

This is because allowing that could break type safety.

For example, if it were allowed:

```
List<Animal> animals = new ArrayList<Dog>();
```

then someone could try:

```
animals.add(new Cat());
```

Now a Cat would enter a list that was originally supposed to contain only Dog.

So Java prevents this.

Later you'll learn:

? extends Animal

and:

? super Dog

to handle such situations safely.

42. Diamond operator <>

You have probably seen:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

What does the empty:

```
<>
```

mean?

It's called the **diamond operator**.

Java can infer the type from the left side.

Instead of:

```
ArrayList<Integer> numbers =  
    new ArrayList<Integer>();
```

we can write:

```
ArrayList<Integer> numbers =  
    new ArrayList<>();
```

Java understands that the right side should use:

Integer

43. Generic class complete example

Let's create a simple StudentBox.

```
class Box<T> {
```

```
    private T value;
```

```
    public Box(T value) {
```

```
        this.value = value;
    }
```

```
    public T getValue() {
        return value;
    }
```

```
    public void setValue(T value) {
        this.value = value;
    }
```

```
    public void display() {
        System.out.println("Value: " + value);
    }
}
```

Main:

```
public class Main {

    public static void main(String[] args) {

        Box<Integer> marks = new Box<>(95);

        System.out.println(marks.getValue());

        Box<String> name = new Box<>("Java");

        System.out.println(name.getValue());

    }
```

```
}
```

Output:

95

Java

44. Let's trace the code

First:

```
Box<Integer> marks = new Box<>(95);
```

Think:

```
Box<T>
```

↓

```
T = Integer
```

Therefore:

```
private T value;
```

means:

```
private Integer value;
```

Constructor:

```
Box(T value)
```

becomes:

```
Box(Integer value)
```

Then:

95

is stored.

Now:

```
Box<String> name = new Box<>("Java");
```

Here:

```
T = String
```

Therefore:

private T value;

becomes conceptually:

private String value;

Same class.

Different type.

That's the main idea behind Generics.

45. Generic method complete example

```
class Utility {
```

```
    public static <T> void display(T value) {  
        System.out.println("Value: " + value);  
    }
```

```
    public static <T> T getValue(T value) {  
        return value;  
    }  
}
```

Use:

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Utility.display(100);
```

```
        Utility.display("Hello");
```

```
        Utility.display(10.5);
```

```
Integer x = Utility.getValue(50);

String y = Utility.getValue("Java");

System.out.println(x);
System.out.println(y);
}
}
```

46. Generic method with arrays

A common example is printing any array.

```
public static <T> void printArray(T[] arr) {

    for (T value : arr) {
        System.out.println(value);
    }
}
```

Use:

```
Integer[] numbers = {10, 20, 30};
```

```
String[] names = {"A", "B", "C"};
```

```
printArray(numbers);
```

```
printArray(names);
```

The same method works with both arrays.

47. Generic method to find an element

Example:

```
public static <T> boolean contains(T[] arr, T target) {
```

```
    for (T value : arr) {
```

```
        if (value.equals(target)) {
```

```
            return true;
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

Use:

```
Integer[] numbers = {10, 20, 30};
```

```
System.out.println(
```

```
    contains(numbers, 20)
```

```
);
```

Output:

true

For String:

```
String[] names = {"Java", "Python", "C++"};
```

```
System.out.println(
```

```
    contains(names, "Python")
```

```
);
```

Output:

true

One generic method works with multiple types.

48. Why .equals() instead of ==?

This connects with something important from Java basics.

For objects such as:

Integer

String

Student

we generally use:

.equals()

to compare their contents.

So:

value.equals(target)

is appropriate when checking object equality.

49. Benefits of Generics

There are several major benefits.

1. Type safety

```
ArrayList<Integer> numbers = new ArrayList<>();
```

Only integers can be added.

2. Compile-time error detection

Wrong types are caught by the compiler.

```
numbers.add("Hello");
```

The compiler complains before execution.

3. No unnecessary casting

Without Generics:

```
Object value = list.get(0);
```

```
Integer x = (Integer) value;
```

With Generics:

```
Integer x = numbers.get(0);
```

4. Code reusability

One class:

```
Box<T>
```

can work with:

Integer

String

Double

Student

Employee

5. Cleaner code

Generic code tells other programmers exactly what type is expected.

```
ArrayList<Student>
```

immediately communicates:

"This list stores Student objects."

50. Generics vs Object

This is worth remembering as an interview comparison.

Generics	Object
Type-safe	Less type-safe
Compile-time checking	Type problems may appear at runtime
Usually avoids explicit casting	Often requires casting
Reusable	Reusable
Clear expected type	Type information is less specific

Example:

Object

Object value = "Hello";

String s = (String) value;

Generic

Box<String> box = new Box<>();

String s = box.getValue();

No manual cast required.

51. Generic Class vs Generic Method — Interview Table

Feature	Generic Class	Generic Method
Type parameter belongs to	Class	Method
Syntax	class Box<T>	<T> void method(T value)
Type decided	When object is created	When method is called
Can have multiple types	Yes	Yes
Can be static	Class itself can't be static at top level	Method can be static
Main purpose	Make class reusable for different types	Make method reusable for different types

52. One very important interview question

Q: Is T a data type?

Not exactly.

T is a **type parameter**.

Example:

class Box<T>

T is a placeholder.

When we create:

Box<String>

the type parameter becomes:

String

When:

Box<Integer>

it becomes:

Integer

53. Is Generic a class?

No.

Generics are a **language feature** in Java that allows classes, interfaces, and methods to work with parameterized types.

For example:

ArrayList<Integer>

is not a new class you manually created.

It is an ArrayList whose type parameter is Integer.

54. Are Generics only for Collections?

No.

You will see them in:

Collections

Classes

Interfaces

Methods

Algorithms

APIs

Frameworks

Collections simply use Generics very heavily.

55. Can a generic class have normal variables too?

Absolutely.

Example:

```
class Box<T> {
```

```
    T value;
```

```
    int id;
```

```
    String name;
```

```
}
```

Here:

T value

is generic.

But:

int id

String name

are normal fixed types.

56. Can a generic class have multiple generic methods?

Yes.

```
class Utility {
```

```
    public <T> void method1(T value) {
```

```
    }
```

```
    public <U> void method2(U value) {
```

```
}  
}
```

Each method can have its own type parameter.

57. Can a generic method exist inside a non-generic class?

Yes!

Very important.

```
class Utility {  
  
    public static <T> void display(T value) {  
        System.out.println(value);  
    }  
}
```

Utility is **not** generic.

But display() is generic.

So:

Generic class → not required

for

Generic method

58. Can a generic class have non-generic methods?

Yes.

```
class Box<T> {  
  
    T value;  
  
    void displayValue() {  
        System.out.println(value);  
    }  
}
```

```
void hello() {  
    System.out.println("Hello");  
}  
}
```

hello() doesn't use T.

That's completely valid.

59. Important mental model

Whenever you see:

<T>

think:

"T is a placeholder for a type."

Whenever you see:

Box<Integer>

think:

"Replace T with Integer for this use of Box."

Whenever you see:

<T> void display(T value)

think:

"This method can work with different types, and Java determines T when I call it."

This mental model will make Generics much easier.

60. Full real-world example: Student Repository

Imagine an application that stores students.

We could create:

```
class Repository<T> {
```

```
private ArrayList<T> items = new ArrayList<>();
```

```
public void add(T item) {  
    items.add(item);  
}
```

```
public T get(int index) {  
    return items.get(index);  
}
```

```
public int size() {  
    return items.size();  
}  
}
```

Now:

```
Repository<String> names = new Repository<>();
```

```
names.add("Ravi");
```

```
names.add("Anu");
```

Or:

```
Repository<Integer> marks = new Repository<>();
```

```
marks.add(90);
```

```
marks.add(85);
```

Or:

```
Repository<Student> students = new Repository<>();
```

```
students.add(student1);
```

```
students.add(student2);
```

One repository class can manage different types.

That's the **real power of Generics**.

61. Generic interface

Generics can also be used with interfaces.

Example:

```
interface Container<T> {
```

```
    void add(T value);
```

```
    T get();
```

```
}
```

Implementation:

```
class Box<T> implements Container<T> {
```

```
    private T value;
```

```
    public void add(T value) {
```

```
        this.value = value;
```

```
    }
```

```
    public T get() {
```

```
        return value;
```

```
    }
```

```
}
```

Use:

```
Container<Integer> container = new Box<>();
```

```
container.add(100);
```

```
System.out.println(container.get());
```

So Generics are not restricted to classes and methods.

62. Generic constructor

A generic class can have constructors that use the class's type parameter:

```
class Box<T> {  
  
    T value;  
  
    Box(T value) {  
        this.value = value;  
    }  
}
```

Here:

T
comes from the class.

63. Generic method inside generic class

This example is important:

```
class Box<T> {  
  
    T value;  
  
    <U> void display(U data) {  
  
        System.out.println(data);  
  
    }  
}
```

```
}
```

There are two type parameters:

$T \rightarrow$ class-level type

$U \rightarrow$ method-level type

Example:

```
Box<Integer> box = new Box<>();
```

```
box.display("Hello");
```

Therefore:

$T = \text{Integer}$

$U = \text{String}$

64. Common mistakes beginners make

Mistake 1

Writing:

```
ArrayList<int>
```

✗

Correct:

```
ArrayList<Integer>
```

Mistake 2

Thinking T means only `String`.

✗

T is just a convention meaning "Type".

Mistake 3

Thinking Generic means only collections.

✗

Generics can be used with classes, methods, interfaces, etc.

Mistake 4

Confusing:

<T>

with:

T

<T> declares the type parameter.

T uses the type parameter.

Example:

<T> void display(T value)

First:

<T>

declares it.

Then:

T value

uses it.

65. One final complete example

Let's combine everything.

```
import java.util.ArrayList;
```

```
class Box<T> {
```

```
    private T value;
```

```
    Box(T value) {
```

```
        this.value = value;
```

```
    }
```

```
T getValue() {  
    return value;  
}
```

```
void setValue(T value) {  
    this.value = value;  
}
```

```
<U> void display(U data) {  
    System.out.println("Data: " + data);  
}  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Box<Integer> box = new Box<>(100);  
  
        System.out.println(box.getValue());  
  
        box.setValue(200);  
  
        System.out.println(box.getValue());  
  
        box.display("Hello");  
  
        box.display(50.5);  
    }  
}
```

```
}  
}
```

Understand the types:

Box<Integer>

↓

T = Integer

But:

box.display("Hello");

has:

U = String

And:

box.display(50.5);

has:

U = Double

So:

Class T → Integer

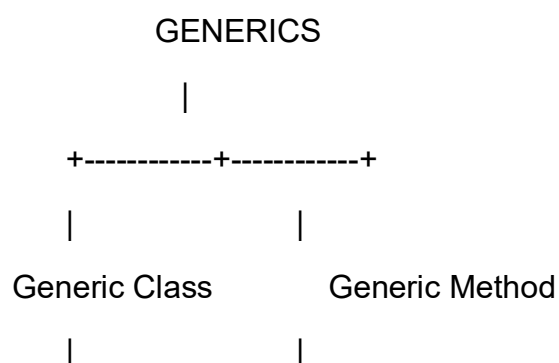
Method U → String

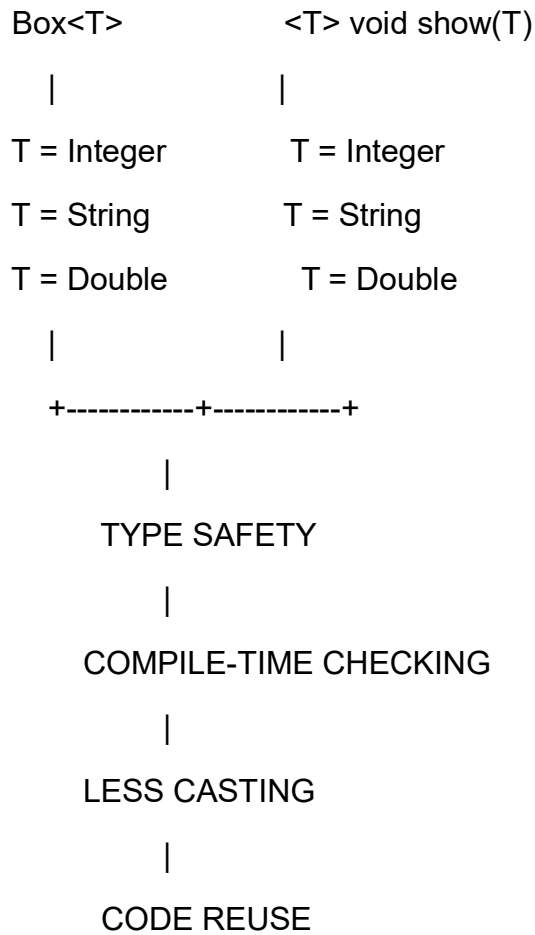
Method U → Double

The class's T and method's U are separate.

66. Generics in one picture

Think about it like this:





67. What you should remember for interviews

If an interviewer asks:

"What are Generics in Java?"

You can answer:

Generics are a Java feature that allows classes, interfaces, and methods to work with parameterized types. They provide compile-time type safety, reduce the need for explicit type casting, and allow code to be reused with different data types.

"What is a generic class?"

A generic class is a class that uses one or more type parameters, such as class Box<T>, allowing the same class to work with different data types.

"What is a generic method?"

A generic method is a method that declares its own type parameter, such as <T> void display(T value), allowing the method to operate on different types independently of the class.

"Why use Generics?"

Remember these four:

TYPE SAFETY



COMPILE-TIME CHECKING



LESS CASTING



CODE REUSABILITY

68. Quick revision sheet

Generic class

```
class Box<T> {
```

```
    T value;
```

```
}
```

Create:

```
Box<Integer> b1 = new Box<>();
```

```
Box<String> b2 = new Box<>();
```

Generic method

```
public static <T> void display(T value) {
```

```
    System.out.println(value);
```

```
}
```

Call:

```
display(10);
```

```
display("Hello");  
display(10.5);
```

Two type parameters

```
class Pair<K, V> {
```

```
    K key;
```

```
    V value;
```

```
}
```

Use:

```
Pair<Integer, String>
```

Collection with Generics

```
ArrayList<Integer>
```

```
ArrayList<String>
```

```
HashSet<Integer>
```

```
HashMap<Integer, String>
```

Primitive types

Don't use:

```
ArrayList<int>    // ✗
```

Use:

```
ArrayList<Integer> // ✔
```

Core idea

```
<T>
```

↓

Type placeholder

Box<Integer>

↓

T becomes Integer

Box<String>

↓

T becomes String

69. Day 34–35 Practice

Now that you've understood the concept, practice these in order.

Level 1 — Basic Generic Class

Create:

```
class Box<T>
```

with:

```
setValue()
```

```
getValue()
```

```
display()
```

Test it with:

Integer

String

Double

Level 2 — Generic Pair

Create:

```
Pair<K, V>
```

Store:

Student ID → Integer

Student Name → String

Example:

101 → "Ravi"

Level 3 — Generic Method

Create:

```
<T> void display(T value)
```

Test it with:

Integer

String

Double

Character

Level 4 — Generic Array Method

Write:

```
<T> void printArray(T[] arr)
```

Test with:

Integer[]

String[]

Double[]

Level 5 — Generic Repository

Create:

```
Repository<T>
```

with:

```
add()
```

```
get()
```

```
remove()
```

```
size()
```

Test:

```
Repository<String>
```

```
Repository<Integer>
```

☆ The one concept you must understand

Don't try to memorize the syntax first.

Understand this:

`class Box<T>`

means:

"I am creating one reusable Box class, but I don't want to decide its data type yet."

Then:

`Box<Integer>`

means:

"For this particular Box, T is Integer."

And:

`Box<String>`

means:

"For this particular Box, T is String."

Similarly:

`<T> void display(T value)`

means:

"This particular method can work with different types, and Java determines the type when the method is called."

Once this **placeholder** → **actual type** idea is clear, almost all basic Java Generics syntax becomes straightforward.